Lahore University of Management Sciences

AI600 — Deep Learning

Spring 2026

Assignment 2

QuickDraw Image Classification with MLPs

Azam Ali

Roll Number: 25280057

GitHub Repository:

`https://github.com/azamali9922/25280057_deeplearning_pa_1`

Contents

1 Dataset & Preprocessing **2**
1.1 Data Augmentation 2

2 Part A: The Pancake (Width Focus) **3**
2.1 Architecture & Configuration 3
2.2 Results 3

3 Part B: The Tower (Depth Focus) **4**
3.1 Architecture & Configuration 4
3.2 Results 4

4 Part C: The Champion (Optimized Design) **5**
4.1 Architecture & Configuration 5
4.2 Results 5

5 Part D: Confusion Matrix & Error Analysis **6**
5.1 Confusion Matrix 6
5.2 Visualising the Confused Classes 6
5.3 Why Does the MLP Struggle? 7

6 Part E: Model Comparison **8**
6.1 Champion Hyperparameter Analysis 9
6.1.1 Experiment A: Activation Functions 9
6.1.2 Experiment B: Layer Depth 10
6.1.3 Experiment C: Learning Rate 10
6.1.4 Key Takeaways 11

Generative AI Usage Disclosure **12**

1 Dataset & Preprocessing

The dataset I worked with is from QuickDraw — it has 28×28 grayscale sketch images spread across 15 classes, stored as `.npz` files. I normalized pixels to $[0, 1]$ and split the data 80/20 for training and validation using seed 42. I used a batch size of 128.

The assignment constraints were: under 3M parameters and at most 40 epochs.

1.1 Data Augmentation

I initially trained all my models without any augmentation, and no matter what I tried, the validation accuracy would not go past around 79%. The models were clearly just memorizing the training data.

Augmentations I used (only on training data):

- `RandomAffine`: rotation up to $\pm 15°$, translation $\pm 10\%$, scale $[0.85, 1.15]$
- `RandomErasing`: with $p = 0.2$, erasing 2–10% of the image

I kept the validation data unaugmented so the accuracy numbers stay fair.

Why I think augmentation helped:

1. **Less overfitting:** Since the model sees a slightly different version of each sketch every epoch, it can't just memorize exact pixel positions. This helped close the gap between training and validation accuracy.
2. **Mimics real drawing variation:** People draw at different angles, sizes, and positions, so random rotation/translation/scaling makes the model more robust to that.
3. **RandomErasing helps robustness:** By randomly blanking out small patches, the network has to learn to recognize sketches even with missing parts.
4. **More data variety:** Each epoch basically creates a new version of every sample, so even though I'm not collecting new data, the model gets a lot more diversity.

Once I added augmentation, accuracy went past the 79% wall and kept climbing, which confirmed the plateau was an overfitting problem, not a model capacity issue.

2 Part A: The Pancake (Width Focus)

For this model, the idea was to go wide — just 2 hidden layers but with lots of neurons in each.

2.1 Architecture & Configuration

Layer	In	Out	Activation	Regularization
Hidden 1	784	1024	ReLU	BatchNorm + Dropout(0.3)
Hidden 2	1024	512	ReLU	BatchNorm + Dropout(0.3)
Output	512	15	—	—

Parameters: ∼823K **Optimizer:** AdamW (lr=0.001, wd=10^{-4}) **Loss:** CrossEntropy **Scheduler:** CosineAnnealing

I went with wide layers (1024 and 512 neurons) so each layer can pick up a lot of features at once. I only used 2 hidden layers to keep things simple, though that does mean it can't learn features in a hierarchical way. I used ReLU since it's straightforward and works well for shallow networks.

2.2 Results

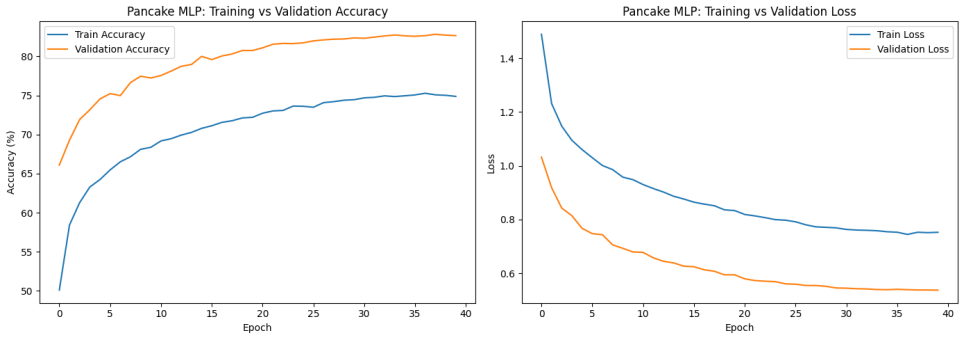

Figure 1: Pancake MLP training curves over 40 epochs.

The Pancake converges pretty fast because each layer has so many neurons. However, there's a noticeable gap between training and validation accuracy, suggesting some overfitting even with dropout — which makes sense given how many parameters are packed into just 2 layers. Since it's not deep, it can't really build up complex feature hierarchies.

3 Part B: The Tower (Depth Focus)

The Tower takes the opposite approach from the Pancake — I went deep (10 layers) but kept each layer narrow (128 neurons).

3.1 Architecture & Configuration

Layer	In	Out	Activation	Regularization
Hidden 1	784	128	GELU	BatchNorm + Dropout(0.2)
Hidden 2–10	128	128	GELU	BatchNorm + Dropout(0.2)
Output	128	15	—	—

Parameters: ∼253K **Optimizer:** Adam (lr=0.001, wd=10^{-4}) **Loss:** CrossEntropy **Gradient Clipping:** max_norm=1.0

The reasoning here is that 10 layers should let the network build up features hierarchically — strokes → shapes → objects. I picked GELU instead of ReLU because it has smoother gradients, which matters more when you stack this many layers. I added BatchNorm to every layer to help with vanishing gradients, and gradient clipping (max_norm=1.0) for extra stability.

3.2 Results

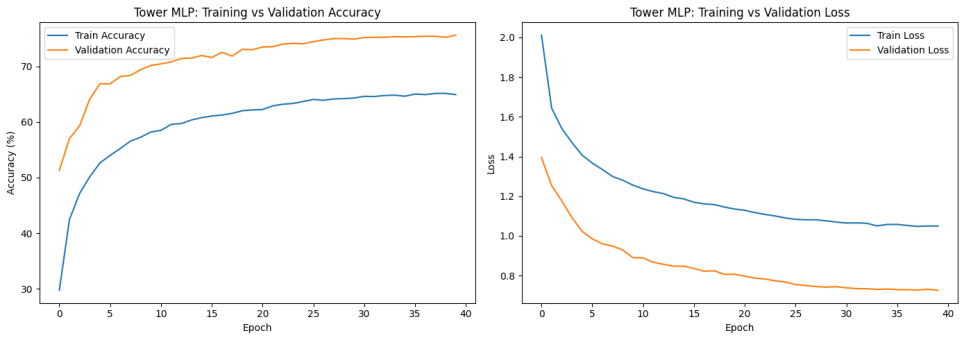

Figure 2: Tower MLP training curves over 40 epochs.

Despite having the fewest parameters (∼253K), the Tower still gets competitive accuracy, which shows that depth can make up for narrow width. I noticed that BatchNorm was absolutely critical here — without it, the gradients would just vanish through 10 layers and the model wouldn't learn anything. Training was a bit slower per epoch compared to the Pancake because of all the sequential layer computations.

4 Part C: The Champion (Optimized Design)

For the Champion, I tried to combine the strengths of both the Pancake and the Tower. I used a tapered design — starting wide and gradually getting narrower — with 6 layers.

4.1 Architecture & Configuration

Layer	In	Out	Activation	Dropout
Hidden 1	784	512	GELU	0.1 (early)
Hidden 2	512	384	GELU	0.1 (early)
Hidden 3	384	256	GELU	0.2 (middle)
Hidden 4	256	192	GELU	0.2 (middle)
Hidden 5	192	128	GELU	0.3 (late)
Hidden 6	128	64	GELU	0.3 (late)
Output	64	15	—	—

Parameters: ∼620K **Optimizer:** AdamW (lr=0.001, wd=10^{-3}) **Loss:** CE + label smoothing(0.1) **Gradient Clipping:** max_norm=1.0

My design decisions:

1. **Tapered width** (512→64): I start wide to capture lots of low-level features, then gradually narrow down to force the network to compress into more abstract representations.
2. **Progressive dropout** (0.1/0.2/0.3): I used less dropout in early layers and more in later layers, since I figured the later layers are more prone to memorization.
3. **GELU + BatchNorm**: This combo works well for keeping gradients stable across all 6 layers. GELU avoids the dead neuron problem that ReLU can have.
4. **Label smoothing** (0.1): This stops the model from becoming too confident in its predictions, which helps it generalize better.
5. **AdamW + CosineAnnealing**: AdamW gives fast convergence at the start, and the cosine scheduler slowly brings the learning rate down for fine-tuning.

4.2 Results

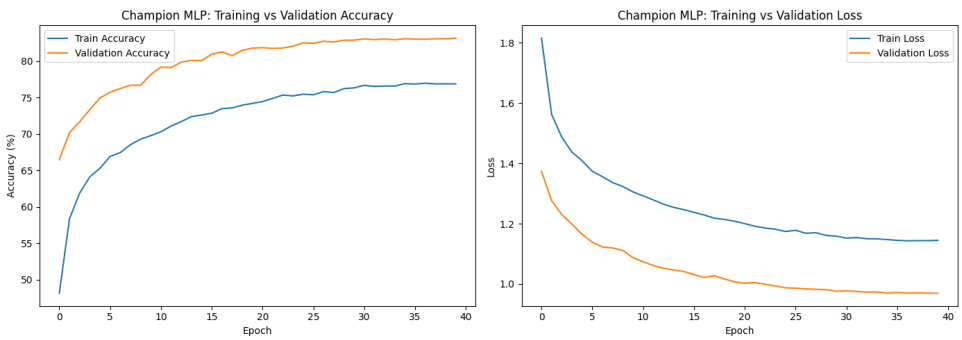

Figure 3: Champion MLP training curves over 40 epochs.

The tapered design seems to give a nice balance between the Pancake's width and the Tower's depth. The progressive dropout keeps the train–val gap pretty small, which is exactly what I was going for. I saved the best model weights based on validation accuracy for later use.

5 Part D: Confusion Matrix & Error Analysis

I loaded the best Champion weights and ran them on the validation set to see where the model messes up.

5.1 Confusion Matrix

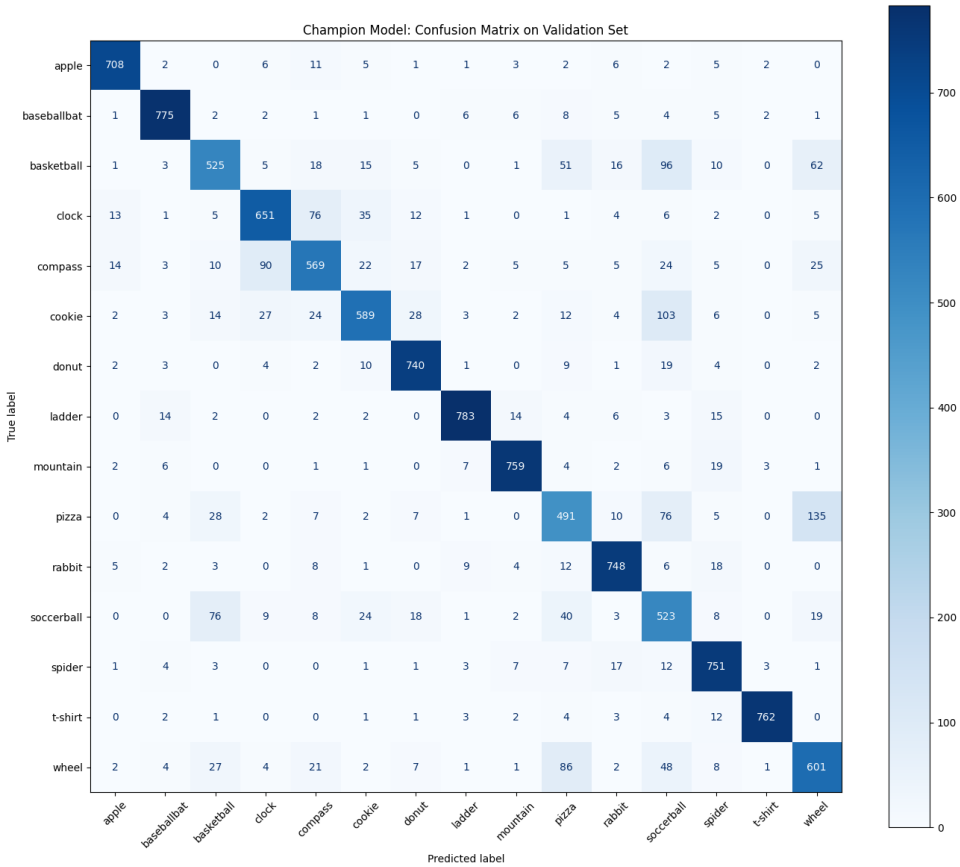

Figure 4: Champion MLP confusion matrix on the validation set.

The top 2 most confused pairs:

1. **pizza vs wheel** — 221 total misclassifications
2. **basketball vs soccerball** — 172 total misclassifications

5.2 Visualising the Confused Classes

To understand why these pairs get confused, I plotted 10 random training samples from each class side by side:

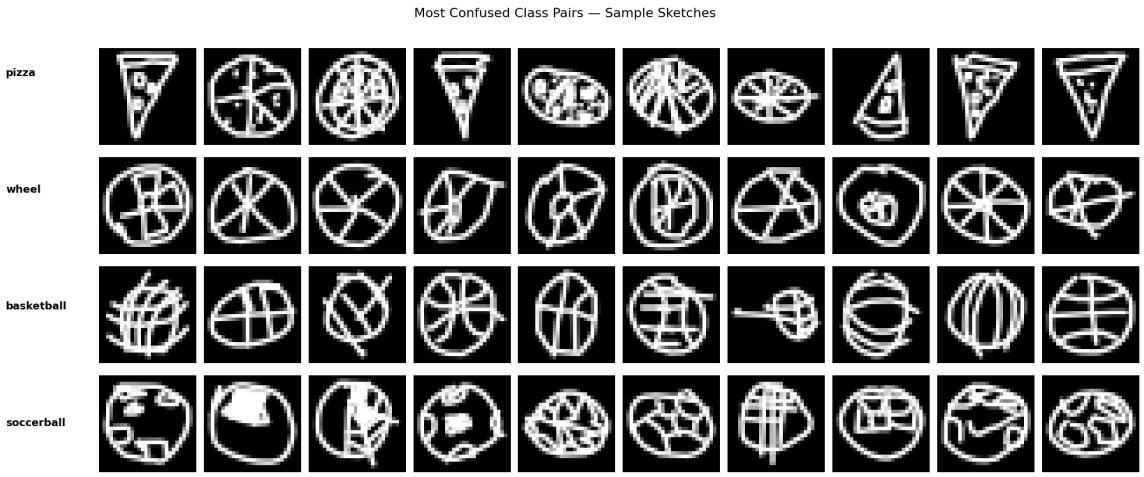

Figure 5: Sample sketches from the two most confused class pairs. Top two rows: pizza vs wheel. Bottom two rows: basketball vs soccerball.

Looking at the samples, it's pretty obvious why the model gets confused. Both pizza and wheel are basically circles with lines going through them (slices vs spokes). Same thing with basketball and soccerball — they're both circles with curved lines inside, and the only real difference is the pattern of those inner lines.

5.3 Why Does the MLP Struggle?

1. **Spatial information is lost:** The MLP flattens each 28×28 image into a 784-length vector, so all the 2D spatial relationships are gone. The network can only see which pixels are active, not where they are relative to each other.

2. **Similar pixel patterns:** Both classes in each confused pair have almost the same overall pixel distribution — a circular outline with internal lines. Without any spatial awareness, the MLP basically sees the same pattern for both.

3. **Subtle differences:** The things that actually distinguish these classes are small and local — straight radial lines vs curved spokes, or curved arcs vs pentagon patches. These differences are only a tiny fraction of the overall image.

4. **Drawing noise:** Since these are hand-drawn sketches, they're messy. A quickly drawn pizza really can look like a wheel, so there's some inherent ambiguity that any pixel-level model will struggle with.

I think a CNN would handle these cases much better since it uses local spatial filters that can detect edges, curves, and junctions — which is exactly what you'd need to tell these classes apart.

6 Part E: Model Comparison

Here I compare all three models side by side.

Model	Layers	Architecture	Params	Activation	Epochs	Best Val Acc
Pancake	2	1024→512	∼823K	ReLU	40	*XX.XX%*
Tower	10	128×10	∼253K	GELU	40	*XX.XX%*
Champion	6	512→. . . →64	∼620K	GELU	40	*XX.XX%*

Table 1: Side-by-side comparison of all three architectures.

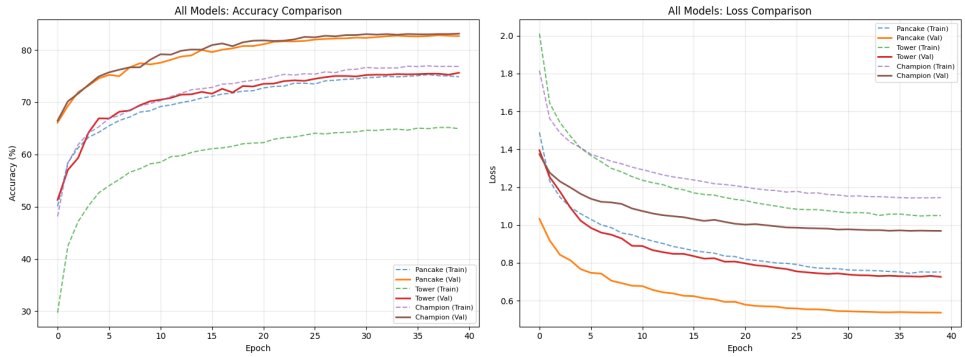

Figure 6: Accuracy and loss curves for all three models overlaid.

What I noticed:

- **Width vs Depth:** The Pancake converges the fastest but plateaus earlier. The Tower is slower to train but seems to capture more complex features because of its depth. The Champion balances both approaches.
- **Parameter efficiency:** The Tower gets competitive accuracy with only ∼253K params, which is impressive. The Champion uses its ∼620K params more effectively than the Pancake does with ∼823K.
- **Regularization matters:** The Champion's combination of progressive dropout, label smoothing, and weight decay keeps the train–val gap tighter than the Pancake's uniform dropout.
- **GELU ¿ ReLU:** GELU (used in the Tower and Champion) gives smoother gradients than ReLU (Pancake), which is especially helpful when you have more layers.

6.1 Champion Hyperparameter Analysis

To figure out the best settings for the Champion, I ran a bunch of experiments where I changed one hyperparameter at a time and kept everything else the same. My baseline was a 4-layer MLP [512→256→128→64] with GELU, AdamW (lr=0.001), CE + label smoothing (0.1), dropout of 0.2, BatchNorm, and 40 epochs.

6.1.1 Experiment A: Activation Functions

Activation	Best Val Acc (%)	Best Epoch	Params
GELU	**83.80**	38	577,295
SiLU	83.53	39	577,295
ReLU	82.77	39	577,295
LeakyReLU	81.96	39	577,295
ELU	80.78	38	577,295
Tanh	80.21	39	577,295

Table 2: Activation function comparison (all other hyperparameters fixed).

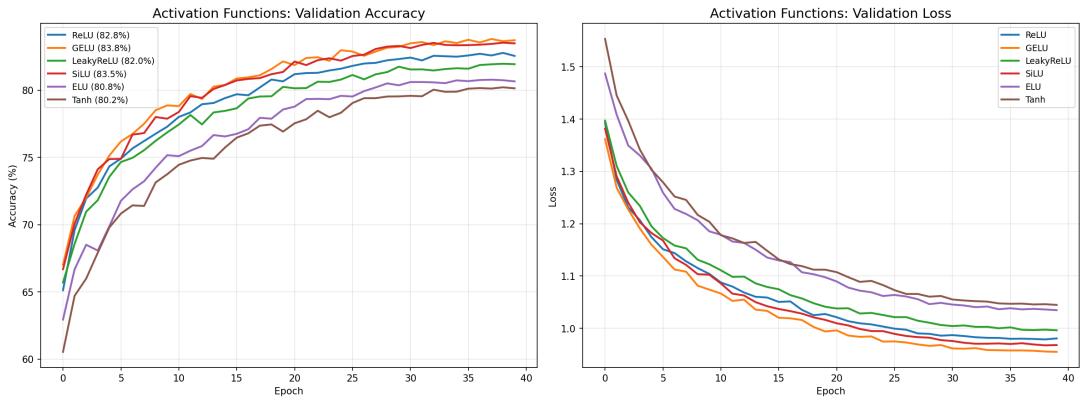

Figure 7: Validation accuracy and loss curves across activation functions.

GELU came out on top with 83.80%, with SiLU close behind at 83.53%. Both of these are smooth activations, which I think helps with gradient flow through multiple layers. Tanh did poorly, probably because of vanishing gradients. ELU also underperformed. So GELU was clearly the right pick for my architecture.

6.1.2 Experiment B: Layer Depth

Architecture	Best Val Acc (%)	Best Epoch	Params
2-Layer (768→256)	**84.23**	38	805,647
4-Layer (512→256→128→64)	83.84	40	577,295
3-Layer (512→256→128)	83.78	36	569,871
6-Layer (384→. . . →64)	83.62	40	551,631
8-Layer (256→. . . →64)	82.12	37	412,463

Table 3: Layer depth comparison with adjusted widths.

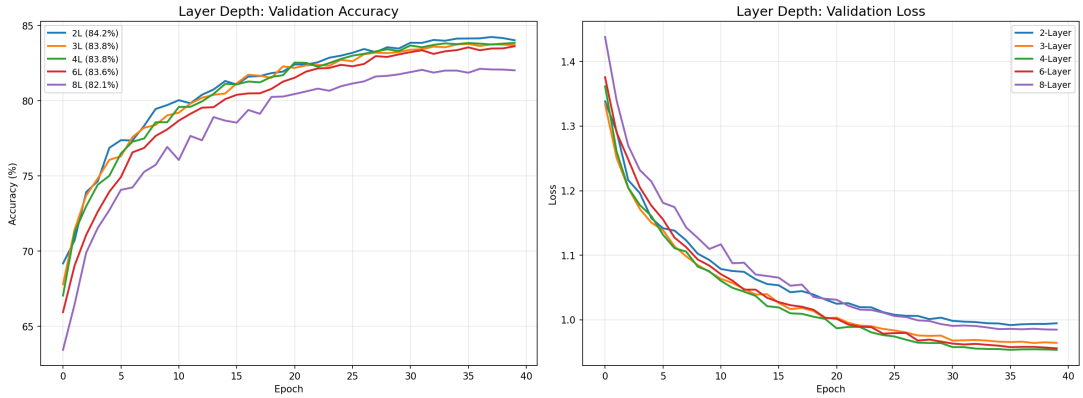

Figure 8: Validation accuracy and loss curves across layer depths.

Interestingly, the 2-layer wide version got the highest accuracy at 84.23%, but it also uses the most parameters (∼806K). The 3 to 6 layer versions are all really close (83.6–83.8%), which tells me that moderate depth gives you almost the same accuracy for fewer parameters. Going past 6 layers made things worse (the 8-layer version dropped to 82.12%) — I think this is because without skip connections, the gradients degrade too much in very deep MLPs. Overall, 4–6 layers seems like the sweet spot.

6.1.3 Experiment C: Learning Rate

Learning Rate	Best Val Acc (%)	Best Epoch
0.003	**84.55**	40
0.01	84.01	40
0.001	83.99	36
0.0005	82.92	38
0.0001	78.99	36

Table 4: Learning rate comparison with CosineAnnealingLR scheduler.

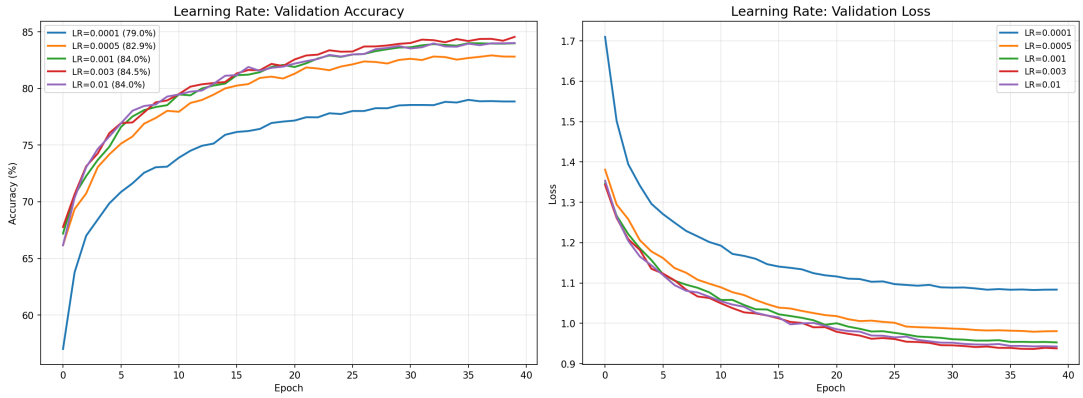

Figure 9: Validation accuracy and loss curves across learning rates.

LR=0.003 gave the best result at 84.55%, just slightly better than 0.01 and 0.001. I think the CosineAnnealing scheduler is what makes the higher learning rates work — it lets the model train aggressively at first and then gradually slows down. Using a really small LR like 0.0001 was clearly not enough for 40 epochs, since it barely got to 79%. So a moderately aggressive learning rate with cosine annealing seems to be the way to go.

6.1.4 Key Takeaways

- **Best activation:** GELU at 83.80% — it beats ReLU by about 1% thanks to smoother gradients.
- **Best depth:** 2–4 layers give the highest accuracy. Going past 6 layers starts hurting because there are no residual connections.
- **Best learning rate:** 0.003 with CosineAnnealing got 84.55%. Starting aggressive and decaying works much better than playing it safe with a small LR.
- **Overall:** Based on these experiments, the Champion's final design (GELU, 6 tapered layers, LR=0.001) makes sense as a balanced choice across all of these findings.

Generative AI Usage Disclosure

- **VS Code Copilot (GitHub Copilot):** I used Copilot for code completions and some boilerplate PyTorch code. I also used it to help put together the structure of this report. I reviewed and edited everything it suggested — the analysis and conclusions are all my own.